QR FACIL

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	QR FACIL
Gerente de Projeto	Guilherme Balbino
Gerente/Responsável de GCS-DevOps	Guilherme Balbino
Fábrica de Software / Squad	Desenvolvimento individual
Repositório principal	https://github.com/GuilhermeBalbino365/qr_facil

Histórico de Revisões

Data	Versão	Descrição	Autor
23/09/2026	1.0	Elaboração do PGCS do projeto QR Fácil.	Guilherme Balbino

Sumário

(No Word, clique com o botão direito sobre esta área e escolha "Atualizar campo" — ou pressione Ctrl+A e F9 — para gerar o sumário automaticamente.)

Sumário 2
1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 4
1.4 Referências 4
1.5 Visão Geral 5
2. Cultura DevOps e Papéis na Gerência de Configuração 5
3. Ferramentas, Ambientes e Infraestrutura 5
3.1 Ferramentas do Pipeline 5
3.2 Ambientes e Infraestrutura 6
3.3 Estratégia de Branching e Versionamento 6
4. Identificação da Configuração 6
4.1 Itens de Configuração 6
4.2 Convenção de Nomenclatura 6
4.3 Estrutura do Repositório 7
5. Integração Contínua (CI) 7
5.1 Etapas do Pipeline de CI 7
5.2 Política de Pull Request e Revisão de Código 7
5.3 Critérios de Qualidade (Quality Gate) 7
6. Entrega e Implantação Contínua (CD) 8
6.1 Etapas do Pipeline de CD 8
6.2 Estratégias de Implantação 8
6.3 Rollback e Recuperação 8
7. Infraestrutura como Código (IaC) 8
7.1 Versionamento de Infraestrutura 8
7.2 Ambientes Efêmeros 8
8. Controle de Configuração e Mudança 9
8.1 Processo de Solicitação de Mudança 9
8.2 Comitê de Controle de Mudança (CCM) 9
9. Baselines e Releases 9
9.1 Geração Automática de Baseline 9
9.2 Notas de Versão (Release Notes) 9
10. Estimativa do Status de Configuração 9
10.1 Monitoramento e Observabilidade 9
10.2 Métricas DORA 9

10.3 Relatórios e Auditorias 10
11. Marcos 10
12. Treinamento e Recursos 10
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 10
14. Quadro-Resumo do Pipeline CI/CD 10

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGCS é destinado ao desenvolvimento individual do projeto QR Fácil, um aplicativo desenvolvido em Flutter/Dart para geração de QR Codes a partir de textos informados pelo usuário. O plano abrange o código-fonte da aplicação, suas dependências, documentação e o repositório Git utilizado para controle e versionamento do projeto.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Repositório do projeto QR Fácil no GitHub: https://github.com/GuilhermeBalbino365/qr_facil

1.5 Visão Geral

O QR Fácil é um aplicativo desenvolvido em Flutter/Dart com o objetivo de gerar QR Codes a partir de textos informados pelo usuário. O aplicativo possui uma interface simples, composta por um campo de entrada de texto, um botão para geração e a exibição do QR Code. A biblioteca externa qr_flutter é utilizada para realizar a geração visual do código. O controle de versão do projeto é realizado por meio do Git e do repositório GitHub, permitindo o registro das alterações realizadas no código-fonte.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Guilherme Balbino — responsável pelo planejamento, acompanhamento e decisões do projeto.
Desenvolvedor(a)	Guilherme Balbino — responsável pelo desenvolvimento, manutenção, versionamento e evolução do aplicativo.
GCS / DevOps	Guilherme Balbino — responsável pelo controle de versão, organização do repositório e gerenciamento das configurações do projeto.
QA / Automação de Testes	Guilherme Balbino — responsável pela realização dos testes e verificação do funcionamento da aplicação.
Segurança (DevSecOps)	Guilherme Balbino — responsável por manter as dependências do projeto atualizadas e evitar o armazenamento de informações sensíveis no repositório.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas sugeridas — substituir pelas efetivamente adotadas pelo projeto:

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento e armazenamento do código-fonte.
Framework	Flutter	Desenvolvimento da aplicação.
Linguagem	Dart	Linguagem utilizada no desenvolvimento.
Geração de QR Code	qr_flutter ^4.1.0	Geração e exibição dos QR Codes.
Testes	flutter_test	Suporte à execução de testes Flutter.
Análise de código	flutter_lints ^6.0.0	Aplicação de regras recomendadas de qualidade e estilo do código.

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Desenvolvimento e testes locais do aplicativo Flutter.	Alterações realizadas no código e testes locais.
Homologação / Staging	Validação da versão do aplicativo antes da apresentação/entrega.	Versão considerada estável após testes.
Produção	Versão final do aplicativo disponibilizada para utilização/apresentação.	Validação da versão de entrega.

3.3 Estratégia de Branching e Versionamento

O projeto utiliza a branch main como branch principal do repositório. Por se tratar de um desenvolvimento individual, as alterações são realizadas diretamente no projeto e registradas por meio de commits no Git. Para futuras evoluções que demandem maior controle de mudanças, poderão ser utilizadas branches de curta duração no padrão feature/<descrição>.

Os commits do projeto seguem uma convenção descritiva, buscando utilizar o padrão Conventional Commits quando aplicável. A versão inicial do projeto é identificada como 1.0.0.

feat: primeira versao do qr_facil

4. Identificação da Configuração

4.1 Itens de Configuração

São considerados itens de configuração do projeto QR Fácil:

• Código-fonte da aplicação Flutter/Dart;
• Arquivo `pubspec.yaml`, contendo as informações e dependências do projeto;
• Código e arquivos relacionados à configuração da aplicação;
• Arquivos de testes, quando existentes;
• Documentação do projeto;
• Arquivos de configuração necessários para execução do aplicativo;
• Repositório Git e histórico de commits.

O projeto utiliza a dependência `qr_flutter`, que deve permanecer registrada no arquivo de dependências e ser versionada juntamente com o projeto.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch principal	main	main

Item	Convenção	Exemplo
Branch de funcionalidade	feature/<descrição>	feature/melhorar-interface
Commit	Conventional Commits	feat: primeira versao do qr_facil
Versão	MAJOR.MINOR.PATCH	1.0.0
Projeto	Nome do projeto	qr_facil

4.3 Estrutura do Repositório

O repositório principal do QR Fácil está hospedado no GitHub e contém o projeto Flutter, organizado de acordo com a estrutura padrão de um aplicativo Flutter. A pasta principal do projeto contém os arquivos de configuração e os diretórios utilizados pelo framework Flutter, incluindo o código-fonte da aplicação e suas dependências. O arquivo principal da aplicação está localizado em **lib/main.dart**.

5. Integração Contínua (CI)

No estado atual do projeto QR Fácil, a integração contínua é realizada de forma manual, por meio da execução e validação local da aplicação antes do envio das alterações ao repositório GitHub. O código é versionado utilizando Git e as alterações são registradas por meio de commits.

5.1 Etapas do Pipeline de CI

No projeto atual, não foi configurado um pipeline de CI automatizado. O processo de validação consiste em:

1. Alteração do código-fonte;
2. Execução da aplicação Flutter localmente;
3. Verificação do funcionamento da geração do QR Code;
4. Correção de eventuais problemas;
5. Registro da alteração por meio de commit Git;
6. Envio da alteração ao repositório GitHub.

5.2 Política de Pull Request e Revisão de Código

Por se tratar de um projeto individual, não há atualmente revisão por pares ou Pull Request obrigatório. As alterações são revisadas pelo próprio desenvolvedor antes do commit e envio para a branch main.

5.3 Critérios de Qualidade (Quality Gate)

Os critérios de qualidade adotados para o projeto são:

• Aplicação deve executar sem erros impeditivos;
• Geração do QR Code deve funcionar corretamente a partir do texto informado;
• Código deve ser mantido versionado no GitHub;

• Dependências devem estar registradas no `pubspec.yaml`;
• Não devem ser armazenadas credenciais ou informações sensíveis no repositório.

6. Entrega e Implantação Contínua (CD)

O projeto QR Fácil não possui atualmente um pipeline automatizado de Entrega/Implantação Contínua (CD). A disponibilização da aplicação é realizada manualmente após a validação da versão desenvolvida.

6.1 Etapas do Pipeline de CD

O processo atual de entrega consiste em:

1. Finalização da alteração;
2. Teste local da aplicação;
3. Validação da geração do QR Code;
4. Registro da versão no Git;
5. Envio do código ao GitHub;
6. Disponibilização da versão validada para apresentação ou utilização.

6.2 Estratégias de Implantação

Não se aplica ao estado atual do projeto. O QR Fácil é um aplicativo Flutter de pequeno porte e não possui atualmente infraestrutura de implantação com múltiplas instâncias ou ambientes automatizados.

6.3 Rollback e Recuperação

A recuperação de versões anteriores é realizada por meio do histórico do Git. As versões anteriores do código permanecem registradas no repositório, permitindo consultar ou restaurar um estado anterior do projeto quando necessário.

7. Infraestrutura como Código (IaC)

A Infraestrutura como Código (IaC) não é utilizada no projeto QR Fácil, pois o aplicativo não possui infraestrutura de servidores ou ambientes de produção provisionados por código. O projeto é executado como uma aplicação Flutter, utilizando o ambiente de desenvolvimento local e as configurações padrão do framework.

7.1 Versionamento de Infraestrutura

Não se aplica, pois não há infraestrutura provisionada por código no projeto.

7.2 Ambientes Efêmeros

Não se aplica ao projeto atual, pois não são utilizados ambientes temporários provisionados automaticamente por Pull Requests.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

As mudanças no QR Fácil seguem o seguinte fluxo:

Identificação da necessidade → alteração do código → teste local → validação da funcionalidade → commit Git → push para o GitHub.

Por se tratar de um projeto individual, não é utilizado atualmente um processo formal de aprovação por Pull Request. O histórico do Git constitui o registro das alterações realizadas.

8.2 Comitê de Controle de Mudança (CCM)

Não há Comitê de Controle de Mudanças formal no projeto QR Fácil, devido ao seu caráter individual e de pequeno porte. Mudanças são analisadas e realizadas pelo próprio desenvolvedor, sendo registradas no histórico do Git.

9. Baselines e Releases

9.1 Geração Automática de Baseline

No estado atual do projeto, as baselines não são geradas automaticamente por pipeline. Uma versão considerada estável pode ser identificada por meio de commits e, quando necessário, por tags no repositório Git.

9.2 Notas de Versão (Release Notes)

As notas de versão não são atualmente geradas automaticamente. As alterações podem ser identificadas por meio das mensagens dos commits registrados no GitHub.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O QR Fácil não possui atualmente monitoramento contínuo de produção, pois é um aplicativo de pequeno porte executado localmente ou em ambiente de apresentação. A validação do funcionamento é realizada por meio de testes da aplicação.

10.2 Métricas DORA

As métricas DORA são apresentadas como referência para práticas DevOps. No estado atual do QR Fácil, não há pipeline de produção automatizado suficiente para coletar essas métricas de forma contínua.

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de commits do Git e o repositório GitHub constituem os principais registros das alterações realizadas no projeto. Por se tratar de um projeto individual, não são utilizados relatórios manuais de auditoria de forma contínua.

11. Marcos

Marcos relacionados ao gerenciamento de configuração do QR Fácil:

• Criação do projeto Flutter;
• Desenvolvimento da funcionalidade de geração de QR Code;
• Inclusão da biblioteca `qr_flutter`;
• Criação do repositório Git;
• Publicação do projeto no GitHub;
• Validação da versão inicial do aplicativo;
• Elaboração e publicação do PGCS.

12. Treinamento e Recursos

O desenvolvimento e manutenção do QR Fácil requerem conhecimentos básicos de Flutter, Dart, Git e GitHub. Para manutenção do projeto, o desenvolvedor deve conhecer a estrutura de aplicativos Flutter, gerenciamento de dependências pelo `pubspec.yaml`, utilização da biblioteca `qr_flutter` e operações básicas de versionamento com Git.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

14. Quadro-Resumo do Pipeline CI/CD

O projeto QR Fácil utiliza a biblioteca de terceiros **`qr_flutter`** para geração dos QR Codes. A dependência é declarada no arquivo **`pubspec.yaml`** e deve ser mantida sob controle de versão juntamente com o projeto. Alterações ou atualizações da dependência devem ser avaliadas antes de sua adoção, verificando a compatibilidade com a versão utilizada do Flutter.

Não são utilizados atualmente softwares desenvolvidos por subcontratados ou fornecedores externos além das bibliotecas declaradas como dependências do projeto.

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
Desenvolvimento	Implementação e alteração do código Flutter.	Necessidade de alteração
Validação	Execução local e teste das funcionalidades.	Finalização da alteração
Commit	Registro da alteração no histórico Git.	Alteração validada
Push	Envio do código para o GitHub.	Commit realizado
Entrega	Disponibilização da versão validada para apresentação/utilização.	Validação concluída